NOTE *Apache Kafka* is an open source distributed event-streaming platform. A *Kafka topic* is like a category to group and organize events or messages. In the previous case, we can see that all events about the Cast data product will land on the team events topic. *Kafka Connect* is a mechanism for connecting a Kafka topic with events from a different technology. In the previous example, we would extract events from the database table and publish them on Kafka.

Kleppmann's pattern is great for building data products in general, but remember that if you are exposing the schema of your database in events, it is not much different from the integration of two systems via the database, but done asynchronously. It is good to transform content into different event schemas that will decouple consumers from the publisher database. It is essential to leave system boundaries even more if a different team owns the consumer.

But let's jump back to architectural drivers of the Cast data product. The team also takes into consideration quality attributes. Security and privacy are ensured on the level of the data product REST API. Compliance with privacy regulations is accomplished by using the privacy regulation compliance enforcer, designed using the enforcer pattern/trait.

DEFINITION The *enforcer pattern, or trait,* is a category or trait of a bounded context. It ensures that other contexts carry out specific operations. In our example, we use components like that to ensure that private user data is deleted when needed.

We also care about data analyst needs. You can use Trino to have SQL access to our Cast database.

DEFINITION *Trino* (https://trino.io/) is a distributed SQL query engine for Big Data. It allows you to query multiple data sources such as SQL databases, MongoDB, Kafka, and S3.

Another option designed by the team can be seen in figure 9.15. We use an Airflow workflow to extract data from the monolith database, transform it, and move it into the Cast Team data product database.